WEEK 01:
Data Preprocessing
• Handling missing values
• Noise detection removal
• Identifying data redundancy and elimination

Code:
```
import pandas as pd
import numpy as np

# Sample dataset
data = {
    'ID': [1, 2, 3, 4, 5],
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [25, np.nan, 35, 40, 30],
    'Salary': [50000, 52000, 58000, 6000000, 59000], # Outlier in 'Salary'
    'Department': ['HR', 'IT', 'HR', 'HR', 'IT']
}

# Creating a DataFrame
df = pd.DataFrame(data)

# Display the original dataset
print("Original Dataset:")
print(df)

# Handling Missing Values
print("\nHandling Missing Values:")
# Option 1: Fill missing values with mean for 'Age'
df['Age'] = df['Age'].fillna(df['Age'].mean())
print("\nAfter filling missing 'Age' with mean:")
print(df)

# Noise Detection and Removal (Outliers in 'Salary')
print("\nNoise Detection and Removal:")
# Calculate IQR to detect outliers in 'Salary'
q1 = df['Salary'].quantile(0.25)
q3 = df['Salary'].quantile(0.75)
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr
# Identify and replace outliers in 'Salary' with NaN
df['Salary'] = np.where((df['Salary'] < lower_bound) | (df['Salary'] >
upper_bound), np.nan, df['Salary'])
```

```
# Fill the missing (noisy) values in 'Salary' with median
df['Salary'] = df['Salary'].fillna(df['Salary'].median())
print("\nAfter handling noisy data in 'Salary':") print(df)

# Identifying and Removing Redundancy
print("\nIdentifying and Removing Redundancy:")
# Check for duplicate rows
duplicates = df.duplicated()
print(f"Duplicate rows detected: {duplicates.sum()}")

# Removing duplicates
df = df.drop_duplicates()
print("\nAfter removing redundancy:")
print(df)

# Final Cleaned Dataset
print("\nFinal Cleaned Dataset:")
print(df)
```

Output:

```
Original Dataset:
   ID     Name   Age   Salary Department
0   1    Alice  25.0    50000         HR
1   2      Bob   NaN    52000         IT
2   3  Charlie  35.0    58000         HR
3   4    David  40.0  6000000         HR
4   5      Eve  30.0    59000         IT

Handling Missing Values:
After filling missing 'Age' with mean:
   ID     Name   Age   Salary Department
0   1    Alice  25.0    50000         HR


1   2      Bob  32.5    52000         IT
2   3  Charlie  35.0    58000         HR
3   4    David  40.0  6000000         HR
4   5      Eve  30.0    59000         IT
```

Noise Detection and Removal:

After handling noisy data in 'Salary':

	ID	Name	Age	Salary	Department
0	1	Alice	25.0	50000.0	HR
1	2	Bob	32.5	52000.0	IT
2	3	Charlie	35.0	58000.0	HR
3	4	David	40.0	55000.0	HR
4	5	Eve	30.0	59000.0	IT

Identifying and Removing Redundancy:
Duplicate rows detected: 0

After removing redundancy:

	ID	Name	Age	Salary	Department
0	1	Alice	25.0	50000.0	HR
1	2	Bob	32.5	52000.0	IT
2	3	Charlie	35.0	58000.0	HR
3	4	David	40.0	55000.0	HR
4	5	Eve	30.0	59000.0	IT

Final Cleaned Dataset:

	ID	Name	Age	Salary	Department
0	1	Alice	25.0	50000.0	HR
1	2	Bob	32.5	52000.0	IT
2	3	Charlie	35.0	58000.0	HR
3	4	David	40.0	55000.0	HR

WEEK 02

Implement any one imputation model

Code:

```
import numpy as np
import pandas as pd
from sklearn.impute import KNNImputer

# Sample dataset with missing values
data = {
    'ID': [1, 2, 3, 4, 5],
    'Age': [25, np.nan, 35, 40, 30],
    'Salary': [50000, 52000, 58000, 60000, np.nan],  # Missing value in 'Salary' 'Department':
    ['HR', 'IT', 'HR', 'HR', 'IT']  # Non-numeric column
}

# Create DataFrame
df = pd.DataFrame(data)

# Display the original dataset
print("Original Dataset:")
print(df)

# Dropping non-numeric columns for imputation numeric_df
= df.drop(columns=['ID', 'Department'])

# Applying KNN Imputation
imputer = KNNImputer(n_neighbors=2)  # Specify number of neighbors imputed_data =
imputer.fit_transform(numeric_df)

# Create a DataFrame with imputed data
imputed_df = pd.DataFrame(imputed_data, columns=numeric_df.columns)

# Adding non-numeric columns back
imputed_df['ID'] = df['ID'] imputed_df['Department']
= df['Department']

# Rearranging columns for clarity
imputed_df = imputed_df[['ID', 'Age', 'Salary', 'Department']]

# Display the dataset after imputation print("\nDataset
After KNN Imputation:") print(imputed_df)
```

Output:

Original Dataset:

	ID	Age	Salary	Department
0	1	25.0	50000.0	HR
1	2	NaN	52000.0	IT
2	3	35.0	58000.0	HR
3	4	40.0	60000.0	HR
4	5	30.0	NaN	IT

Dataset After KNN Imputation:

	ID	Age	Salary	Department
0	1	25.0	50000.0	HR
1	2	30.0	52000.0	IT
2	3	35.0	58000.0	HR
3	4	40.0	60000.0	HR
4	5	30.0	54000.0	IT

3.Implement Linear Regression

```
# Step 1: Generate some data points
x = [1, 2, 3, 4, 5] # Independent variable
y = [2, 4, 5, 4, 5] # Dependent variable
# Step 2: Calculate means of x and y
mean_x = sum(x) / len(x)
mean_y = sum(y) / len(y)
# Step 3: Calculate slope (m) and intercept (b)
numerator = sum([(x[i] - mean_x) * (y[i] - mean_y) for i in range(len(x))])
denominator = sum([(x[i] - mean_x) ** 2 for i in range(len(x))])
m = numerator / denominator
b = mean_y - m * mean_x
# Step 4: Predict y values using the model
y_pred = [m * xi + b for xi in x]
print("Slope (m):", m)
print("Intercept (b):", b)
print("Predicted y values:", y_pred)
```

OUT PUT

```
Slope (m): 0.6
Intercept (b): 2.2
Predicted y values: [2.8000000000000003, 3.4000000000000004, 4.0, 4.6, 5.2]
```

4.Implement Logistic Regression

```
# Step 1: Generate binary classification data
x = [1, 2, 3, 4, 5] # Independent variable
y = [0, 0, 0, 1, 1] # Dependent variable (binary classification)

# Step 2: Sigmoid function
import math # import math module to use math.exp()
def sigmoid(z):
    return 1 / (1 + math.exp(-z)) # Indent the return statement to be part
of the function

# Step 3: Initialize parameters
m, b = 0, 0 # Start with initial values
learning_rate = 0.01
num_iterations = 1000

# Step 4: Gradient Descent for Logistic Regression
for _ in range(num_iterations):
    m_grad = sum([(sigmoid(m * x[i] + b) - y[i]) * x[i] for i in
range(len(x))])
    b_grad = sum([(sigmoid(m * x[i] + b) - y[i]) for i in range(len(x))])

    # Update the parameters

    m -= learning_rate * m_grad
    b -= learning_rate * b_grad

# Step 5: Predict probabilities using the sigmoid function
y_pred_prob = [sigmoid(m * xi + b) for xi in x]

# Convert probabilities to class predictions (0 or 1)
y_pred = [1 if prob > 0.5 else 0 for prob in y_pred_prob]

print("Optimized Slope (m):", m)
print("Optimized Intercept (b):", b)
print("Predicted class probabilities:", y_pred_prob)
print("Predicted classes:", y_pred)
OUT PUT:
Optimized Slope (m): 1.2693567720287338
Optimized Intercept (b): -4.177149454816769
Predicted class probabilities: [0.05176968460638721, 0.16267800083744785,
0.40876355284120997, 0.7110065530114915, 0.8974893359190727]
```

```
Predicted classes: [0, 0, 0, 1, 1]
```

5.Implement Decision Tree Induction for classification

```
# Import necessary libraries
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# Load the Iris dataset
iris = load_iris()
X = iris.data          # Features (flower measurements)
y = iris.target        # Labels (species)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Create the Decision Tree Classifier
model = DecisionTreeClassifier()

# Train the model on the training data
model.fit(X_train, y_train)

# Predict the species of test data
y_pred = model.predict(X_test)

# Check the accuracy of the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy * 100:.2f}%")

# Visualize the Decision Tree
plt.figure(figsize=(12,8))
plot_tree(model, filled=True, feature_names=iris.feature_names,
class_names=iris.target_names)
plt.show()
```

OUTPUT

```
Model Accuracy: 100.00%
```

6.Implement Random Forest Classifier

```
# Import required libraries
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Step 1: Load the dataset
iris = load_iris()
X = iris.data          # Features (like petal length, width, etc.)
y = iris.target        # Target (species of flowers)

# Step 2: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Step 3: Create the Random Forest Classifier
model = RandomForestClassifier(n_estimators=100, random_state=42)

# Step 4: Train the model
model.fit(X_train, y_train)

# Step 5: Make predictions on the test data
y_pred = model.predict(X_test)

# Step 6: Calculate and print the accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Random Forest Model Accuracy: {accuracy * 100:.2f}%")

OUTPUT:

Random Forest Model Accuracy: 100.00%
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
```

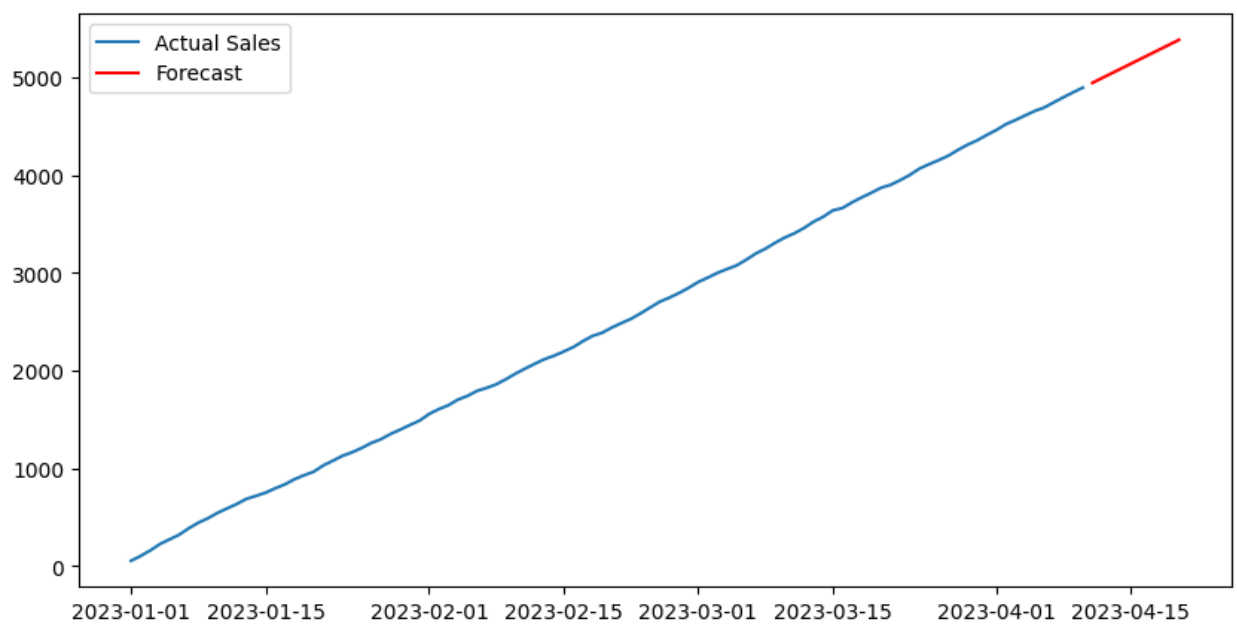
```
# Generate Sample Time Series Data
np.random.seed(42)
dates = pd.date_range(start='2023-01-01', periods=100, freq='D')
sales = np.cumsum(np.random.randn(100) * 10 + 50)  # Simulated Sales Data
data = pd.Series(sales, index=dates)

# Fit ARIMA Model (p=1, d=1, q=1)
model = ARIMA(data, order=(1,1,1))
model_fit = model.fit()

# Forecast Next 10 Days
forecast = model_fit.forecast(steps=10)

# Plot Data & Forecast
plt.figure(figsize=(10,5))
plt.plot(data, label="Actual Sales")
plt.plot(pd.date_range(start=dates[-1], periods=11, freq='D')[1:],
forecast, label="Forecast", color='red')
plt.legend()
plt.show()
```

OUT PUT



8.Object segmentation using hierarchical based methods

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load Image in Grayscale
image_path = r"C:\Users\hp\Desktop\OIP.jpeg"
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

# Check if the image is loaded
if image is None:
    raise FileNotFoundError(f"Error: Unable to load image at {image_path}")

# Resize image to speed up processing (Optional)
image = cv2.resize(image, (200, 200))

# Reshape image for clustering
pixels = image.reshape(-1, 1).astype(np.float32)

# Apply K-Means Clustering (Faster than Hierarchical)
num_clusters = 3
_, labels, centers = cv2.kmeans(pixels, num_clusters, None,
                                (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 10,
1.0),
                                10, cv2.KMEANS_RANDOM_CENTERS)

# Reshape labels back to image dimensions
segmented = labels.reshape(image.shape)

# Display Images
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(segmented, cmap="jet")
plt.title(f"Segmented Image ({num_clusters} Clusters)")
plt.axis("off")

plt.show()
```